Studio Nx

Backend(FastAPI/Python) 22 — POST /api/v1/auth/signup

2026-10-07

Backend(FastAPI/Python) 22 — 라우터 `POST /api/v1/auth/signup`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `signup`)
작업	P1-04 회원가입 — 가입 API · PRD 01 FR-101 · §7
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	21 `signup()` · 23 `GET /auth/check-email`

1. 하는 일

브라우저가 보낸 가입 요청을 받아 21 `auth_service.signup()` 에 넘기고, 결과를 **응답 모양**으로 바꿔 돌려준다.

```
POST /api/v1/auth/signup
{"email": "Route.User@Example.com", "nickname": "route_user", "password": "Abcdef1!",
 "privacy_agreed": true, "privacy_version": "2026-10-05"}

→ 201 Created
{"user_id": 92, "email": "route.user@example.com", "nickname": "route_user"}
```

2. 언제 쓰나

join.html 의 **Sign Up** 버튼 → `join.js`(P1-07) 가 이 주소로 보낸다. 201 을 받으면 화면은 `home.html#login` 으로 옮겨 로그인 모달을 연다(가입 후 자동 로그인은 하지 않음).

라우터 · 서비스는 무엇이 다른가

층	파일	하는 일
라우터 (이 함수)	`api/v1/auth.py`	주소 · 방식(`POST`) · 응답 상태(201) · 응답 모양을 정함. HTTP 를 아는 층
서비스 (21)	`services/auth_service.py`	실제 검사 · 저장. HTTP 를 모름 — 시험이나 다른 곳에서도 그대로 부를 수 있음

요청 하나가 지나가는 길

순서	누가	일어나는 일
①	FastAPI	본문 JSON 을 `SignupRequest` 와 비교. 칸이 빠졌거나 모양이 틀리면 **라우터 실행 전에** 422 (02 처리기)
②	FastAPI	`db: Db` → `get_db()` 로 DB 세션을 엶
③	**이 함수**	`auth_service.signup(db, data)`
④	21	검사 · 저장. 실패하면 `AppError` → 01 처리기가 409 · 422 응답
⑤	**이 함수**	`User` → `SignupResponse` (번호 · 이메일 · 닉네임만)
⑥	FastAPI	201 + JSON. 요청이 끝나면 `get_db()` 가 세션을 닫음

결과	상태	예
가입됨	201	`{"user_id": 92, …}`
칸 빠짐 · 모양 틀림	422	`details: {"privacy_agreed": "Field required"}`
비밀번호 규칙 · 동의	422	`details: {"password": ["upper", …]}`
이메일 · 닉네임 사용 중, 탈퇴 3일 안	409	`code: "EMAIL_TAKEN"` 등

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다 (P1-05 로그인 것까지 함께 있음). **`from app.services import auth_service` 를 이번에 추가했다.** 이 함수는 `APIRouter`(→ `router`) · `status` · `Db` · `SignupRequest` · `SignupResponse` · `ErrorResponse` · `auth_service` 를 쓴다.

import	쓰는 곳
`router = APIRouter()`	`@router.post(…)` — 이 파일의 주소들을 묶음. `api/v1/router.py` 가 `/auth` 아래에 붙임
`status.HTTP_201_CREATED`	성공 상태 201
`Db`	DB 세션 (P1-03 `deps.py`)
`SignupRequest` · `SignupResponse`	요청 · 응답 모양
`ErrorResponse`	API 문서(`/docs`)에 409 · 422 응답 모양을 보여 줌
`auth_service`	21 의 `signup()` 을 `auth_service.signup` 으로 부름

함수

```
@router.post(
    "/signup",
    response_model=SignupResponse,
    status_code=status.HTTP_201_CREATED,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def signup(data: SignupRequest, db: Db) -> SignupResponse:
```

```
    user = auth_service.signup(db, data)
    return SignupResponse(user_id=user.user_id, email=user.email, nickname=user.nickname)
```

위의 `@router.post(…)` 부분은 템플릿에 있었고, 함수 몸통 두 줄을 이번에 채웠다.

문법 풀이

import 문법	뜻
`from app.services import auth_service`	**모듈(파일) 전체**를 이름 하나로 가져옴 → `auth_service.signup` 처럼 씀. 이 파일의 라우터 함수 이름도 `signup` 이라, 서비스 함수를 `from … import signup` 으로 가져오면 이름이 겹친다

식	뜻
`@router.post("/signup", …)`	데코레이터 — 아래 함수를 `POST /signup` 요청에 연결
`response_model=SignupResponse`	응답을 이 모양으로 **걸러서** 보냄. 다른 칸이 있어도 나가지 않음
`status_code=201`	성공하면 200 이 아니라 201(만들어짐)
`responses={409: …, 422: …}`	응답 설명을 API 문서에만 추가 (동작에는 영향 없음)
`data: SignupRequest`	요청 본문 JSON 을 이 모양으로 읽어 넣어 줌 (①)
`db: Db`	DB 세션을 넣어 줌 (②)

4. 흐름

1. FastAPI 가 본문을 검사하고 DB 세션을 연다.
2. 21 `signup()` 으로 가입한다.
3. 번호 · 이메일 · 닉네임만 담아 201 로 응답한다.

5. 왜 이렇게

선택	이유
라우터는 두 줄	검사 · 저장은 서비스(21)에 둔다. 라우터가 얇으면 HTTP 없이도 서비스를 시험할 수 있다
`SignupResponse` 로 바꿔서 응답	`User` 를 그대로 내보내면 `password_hash` 같은 칸이 섞일 위험이 있다. 응답 모양을 정해 두면 그 칸만 나간다 (PRD 01 §5)
201	REST 관례 — 새로 만들었다는 뜻. 화면은 201 로 "가입 성공" 을 안다
오류는 여기서 잡지 않음	21 이 던진 `AppError` 를 01 처리기가 받아 모양을 맞춘다. 라우터마다 `try` 를 쓰지 않아도 된다
가입 후 토큰을 주지 않음	로그인은 P1-05 의 일이다. 가입과 로그인을 나눠 두면 각자 단순하다

6. 시험 (`tests/test_auth_routes.py` 에 4개 추가 · 모두 6개, 실제 앱 + 실제 MySQL · rollback)

시험에서 앱의 `get_db` 를 시험용 세션으로 바꿔 끼워(`app.dependency_overrides`), 진짜 주소로 요청했다.

시험	확인한 것
가입	201, 응답 칸이 정확히 `user_id` · `email` · `nickname` (비밀번호 · 해시 없음), 이메일 소문자, DB 에 있음
같은 이메일로 다시	두 번째는 409 `EMAIL_TAKEN`

시험	확인한 것
비밀번호 규칙	422, `details.password = ["upper", "number", "special"]`
칸 빠짐	422, `details = {"privacy_agreed": "Field required"}` (02 처리기)

전체 시험 149개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.